Feature `Secure Account Registration and Authentication`

User Story 13: Register an Employee Account

US 13 As a new employee, I want to register an Employee account so that I can use the URL analysis workspace without depending on a pre-created account.

Acceptance Criteria

1. Given a visitor has no active session, when they open the ClearURL landing page, then the authentication card displays both `Sign in` and `Register` options.
2. Given the visitor selects `Register`, when the registration form is shown, then it requests a full name, email address, password, password confirmation, graphical CAPTCHA code, and a submit action.
3. Given the visitor enters different password and confirmation values, when they submit the registration form, then the frontend prevents submission and displays `Passwords do not match.`
4. Given the visitor enters a name that is blank or longer than 80 characters, when the registration request reaches the backend, then the backend rejects the request and returns a readable display-name validation error.
5. Given the visitor enters an email address without an `@` character or longer than the supported email length, when they submit registration, then the backend rejects the request and does not create an account.
6. Given the visitor enters a password shorter than 10 characters, when they submit registration, then the backend rejects the request with the password requirement and does not store the new account.
7. Given the visitor enters an email already stored in the authentication database, when they submit registration, then the backend rejects the duplicate request and does not create a second account.
8. Given all registration fields and the CAPTCHA response are valid, when the registration succeeds, then the backend creates a user record with the `employee` role and `is_active` status set to true.
9. Given registration succeeds, when the backend returns the new session, then the frontend stores the session and opens the Employee analysis workspace.

10. Given a visitor uses the public registration form, when an account is created, then the role is always `employee`; the request cannot create an `admin` account.
11. Given registration fails after a CAPTCHA has been submitted, when the failure is displayed, then the frontend requests a replacement CAPTCHA because the submitted challenge is single-use.

List

- Provide `Sign in` and `Register` tabs in the authentication interface.
- Validate password confirmation in the browser before calling the registration API.
- Validate display name, email, password length, CAPTCHA, and duplicate emails in the backend.
- Store new user records in `data/auth.sqlite3`.
- Create an Employee session only after successful account creation.

User Story 14: Complete Graphical CAPTCHA Verification

US 14 As a user signing in or registering, I want to enter a graphical verification code so that basic automated login and account-creation attempts are discouraged.

Acceptance Criteria

1. Given the user is on either the `Sign in` or `Register` form, when the form is displayed, then ClearURL requests a CAPTCHA challenge from the authentication API.
2. Given the CAPTCHA API responds successfully, when the frontend receives the response, then it displays the returned verification image and an input field labelled `Enter the code shown above`.
3. Given the user cannot read the image or wants a replacement, when they select `New image`, then the frontend requests a new challenge and clears the previously entered code.
4. Given a CAPTCHA is generated, when the backend creates it, then the code is generated server-side from a non-ambiguous character set and returned only as a rendered SVG data image rather than a fixed frontend answer.
5. Given a user submits sign-in or registration without a CAPTCHA ID or answer, when the backend receives the request, then it rejects the request with `CAPTCHA_REQUIRED`.

6. Given a user submits a CAPTCHA answer, when the answer does not match the server-side challenge, then the backend rejects the request with `CAPTCHA_FAILED`.
7. Given a CAPTCHA was issued more than five minutes earlier, when the user submits it, then the backend treats it as expired and rejects the request.
8. Given a CAPTCHA is submitted once, whether the answer is correct or incorrect, when the backend validates it, then the challenge is removed and cannot be reused.
9. Given sign-in or registration fails after a CAPTCHA submission, when the frontend receives the error, then it requests a new verification image for the next attempt.
10. Given a CAPTCHA is correct but credentials are invalid, when the login request is processed, then the login still fails with the credential error; CAPTCHA validation does not bypass account authentication.
11. Given a CAPTCHA is correct and the login or registration fields are valid, when the server completes validation, then the user can proceed to session creation.

List

- Serve `GET /api/auth/captcha` from the authentication API.
- Render the returned CAPTCHA image in both authentication flows.
- Send `captchaId` and `captchaAnswer` in sign-in and registration payloads.
- Store challenges in backend memory with expiry and single-use behaviour.
- Refresh the image after failed authentication attempts.

Feature `Administrator Account Access Control`

User Story 15: Manage Employee Account Access

US 15 As an admin, I want to review registered accounts and suspend or restore Employee access so that I can control who is allowed to use ClearURL.

Acceptance Criteria

1. Given an authenticated Admin opens the administration console, when they select `Account management`, then the workspace displays the registered accounts list.
2. Given the account list is loaded, when each row is displayed, then it includes the display name, email, role, current status, join date, and latest recorded session date

when one exists.

3. Given an Employee account has `is_active = true`, when an Admin selects `Suspend access`, then the backend updates that account to inactive and returns the updated account status.
4. Given an Employee account has `is_active = false`, when an Admin selects `Restore access`, then the backend updates that account to active and returns the updated account status.
5. Given an Employee account is suspended, when the status change is stored, then all sessions belonging to that account are deleted from the session table.
6. Given a suspended Employee attempts to sign in, when their credentials are otherwise correct, then the backend rejects access with an account-suspended message.
7. Given a suspended Employee has an old stored session token, when they call an authenticated API or the frontend restores the session, then the backend rejects the token and removes the invalid session.
8. Given the account list is displayed, when an account is active, then the interface shows an `Active` status badge; when it is suspended, it shows a `Suspended` status badge.
9. Given an account has the `admin` role, when it appears in Account management, then it is marked `Protected` and no suspend or restore control is provided.
10. Given an Admin attempts to change their own account status or the status of another Admin through the API, when the request is processed, then the backend rejects the change.
11. Given a non-admin user requests the account list or account-status endpoint, when the request is processed, then the backend returns an authorization error and does not expose account data.

List

- Add `Account management` to the Admin navigation.
- Read account data from the authentication database.
- Add active-status migration support for existing user tables.
- Provide authenticated Admin APIs to list accounts and update Employee status.
- Remove Employee sessions when access is suspended.

User Story 16: Use the Admin Console on Smaller Screens

US 16 As an admin using a smaller screen, I want the Admin navigation and sign-out control to remain usable so that I can securely manage ClearURL without hidden or overflowing actions.

Acceptance Criteria

1. Given the Admin console is viewed on a desktop-width screen, when the page loads, then the navigation is shown as a left-hand sidebar and the current view appears in the workspace to its right.
2. Given the desktop sidebar contains more content than the viewport height, when the user scrolls within the sidebar, then all navigation entries and the account footer remain reachable.
3. Given the viewport becomes narrower than the responsive desktop breakpoint, when the Admin console reflows, then the sidebar becomes a full-width navigation area above the workspace.
4. Given the responsive navigation is shown, when the user views the available entries, then the entries are arranged in a multi-column grid rather than being clipped horizontally.
5. Given the responsive navigation is shown, when the page is scrolled, then the signed-in administrator name, role, and `Sign out` button remain inside the Admin navigation area rather than overflowing into the site footer.
6. Given the Admin selects `Sign out` at any supported viewport size, when the logout request succeeds or fails, then the frontend clears local session state and returns to the unauthenticated landing page.
7. Given the current Admin version is displayed, when the user reviews navigation entries, then `Feedback insights` is not present because this incomplete view has been removed from the Iteration 2 interface.

List

- Keep the desktop sidebar vertically scrollable.
- Reflow Admin navigation into responsive grid layouts at smaller widths.
- Preserve the Admin footer and logout action in responsive layouts.
- Remove the `Feedback insights` page, navigation entry, and frontend data request.

Feature `Employee Feedback and Reviewed URL Policy`

User Story 17: Request a Result Review or Report a Risk

US 17 As an employee, I want to report a suspected threat or request a review of a possible misclassification so that an administrator can assess the URL and improve future handling.

Acceptance Criteria

1. Given an Employee has received a URL analysis result, when the result is displayed, then a `Send for review` action is available.
2. Given the Employee selects `Send for review`, when the review form opens, then it displays the current system risk category so the user understands what they are responding to.
3. Given the review form is open, when the Employee selects the report type, then they can choose either to report suspected risk or request a result review.
4. Given the Employee reports suspected risk, when they select a reason, then the form provides structured reasons such as credential requests, unexpected messages, brand impersonation, domain concerns, or another reason.
5. Given the Employee requests a review, when they select a reason, then the form provides structured reasons such as official-service recognition, trusted internal use, an overly strong warning, underestimated risk, normal expected content, or another reason.
6. Given the Employee wants to add context, when they enter a note, then the note is optional and limited to 400 characters.
7. Given the Employee submits the review form, when the backend accepts it, then the request ID, normalized URL, review type, Employee assessment, reason code, optional note, and timestamp are stored.
8. Given the same normalized URL is reported more than once, when feedback is stored, then ClearURL aggregates the reports into the same reviewable case rather than creating uncontrolled duplicate cases.
9. Given review submission succeeds, when the frontend receives the success response, then it displays the submitted state and prevents accidental duplicate submission from the same result screen.
10. Given review submission fails, when the backend or network returns an error, then the Employee remains on the review form and receives an error message without losing the entered note.

List

- Provide a review form from the Employee result screen.
- Collect structured report/review reasons and optional notes.
- Persist feedback in the feedback database.
- Aggregate repeated reports by normalized URL.
- Display aggregated review cases in the Admin case queue.

User Story 18: Apply a Reviewed Allowlist or Blocklist Decision

US 18 As an admin, I want to apply a documented allowlist or blocklist decision to a reviewed URL so that future scans can use a human-approved policy before normal model inference.

Acceptance Criteria

1. Given an authenticated Admin opens a case from `Review cases`, when the case review dialog is displayed, then the dialog shows the URL, current case status, report count, and any existing administrator note.
2. Given the Admin decides the URL is trusted, when they select `Add to allowlist`, then the dialog requests a decision reason and offers expiry choices of 1, 7, 30, or 90 days, or permanent.
3. Given the Admin decides the URL is confirmed risky, when they select `Add to blocklist`, then the dialog requests a decision reason and does not offer a temporary expiry option.
4. Given the Admin has not entered a decision reason, when they attempt to continue, then the frontend prevents the update and explains that a review reason is required.
5. Given the Admin has supporting evidence, when they attach a website screenshot, then the system stores the optional screenshot name and data reference with the decision record.
6. Given the Admin reviews the decision summary, when they select the final confirmation action, then the backend stores the decision type, URL, reason, administrator identity, timestamp, expiry information, and optional evidence metadata.
7. Given an active allowlist entry exists for the exact normalized URL, when an Employee submits that URL again, then the policy result is applied before normal model inference.
8. Given an active blocklist entry exists for the exact normalized URL, when an Employee submits that URL again, then the blocklist policy result is applied before

normal model inference.

9. Given a temporary allowlist entry reaches its expiry time, when the same URL is scanned later, then it no longer overrides the model; the expired entry remains visible in the policy list for audit.
10. Given an Admin opens `Policy lists`, when they search by URL or decision reason and optionally filter by list type, then matching allowlist and blocklist entries are displayed.
11. Given an Admin removes a policy entry, when the removal succeeds, then future scans of that URL return to the normal analysis flow unless another active policy applies.
12. Given a non-admin user calls a policy or case-action API, when the request is processed, then the backend rejects the request with an authorization error.

List

- Provide allowlist/blocklist controls and confirmation in the case-review dialog.
- Store policy decisions and expiry information in the feedback database.
- Apply active policy entries before model analysis.
- Display searchable policy records with active, expired, and permanent status.
- Allow Admin users to remove obsolete policy entries.

Feature `Explainable Identity Evidence and Operational Monitoring`

User Story 19: Inspect Brand-Impersonation Evidence

US 19 As an admin, I want to inspect recorded brand-impersonation evidence so that I can recognise lookalike domains and character-substitution attacks during an investigation.

Acceptance Criteria

1. Given an authenticated Admin selects `Brand visualiser`, when the view loads, then it displays a summary of recorded brand-impersonation observations.

2. Given a scan has domain-identity evidence, when it is listed, then the view shows the observed domain and the monitored official target domain.
3. Given the observed domain is similar to a monitored official domain, when the evidence is displayed, then the view shows the calculated similarity score.
4. Given a lookalike uses character substitution, such as `0` in place of `o`, when the substitution is detected, then the view identifies the observed and expected characters and their position.
5. Given a brand name is placed outside the official registrable domain, when this pattern is detected, then the view labels the deceptive brand-placement technique.
6. Given the URL uses an `@` character to obscure the effective destination domain, when this pattern is detected, then the view labels the destination-obfuscation technique.
7. Given an item is shown, when the Admin reviews it, then the recorded risk category and safety score are displayed for operational context.
8. Given a website could not be fetched, when its scan is represented in this view, then the interface does not describe unreachable status alone as proof that the URL is phishing.
9. Given there are no recorded scans with brand-impersonation evidence, when the view loads, then it displays a clear empty-state message rather than a blank panel.

List

- Extract domain-identity and lookalike evidence during URL feature processing.
- Record evidence with scan data.
- Expose recorded evidence through an authenticated Admin API.
- Present domain comparisons, substitutions, and techniques in the Brand visualiser.

User Story 20: Review Model Quality and Live Operations

US 20 As an admin, I want to review held-out model evaluation and live system operations separately so that I do not confuse model quality with current scan activity.

Acceptance Criteria

1. Given an authenticated Admin selects `Model monitoring`, when recorded evaluation data exists, then the view displays the model algorithm, model version, and dataset version.

2. Given evaluation metrics are available, when the view is displayed, then it shows accuracy, precision, recall, F1 score, ROC-AUC, false positive rate, false negative rate, and average inference time.
3. Given the system has recorded scans and cases, when the live operations panel is displayed, then it shows total scans, open Employee review cases, average API response time, and the count of active policy overrides.
4. Given the Admin compares the two panels, when they read the descriptions, then held-out evaluation is identified as separate from operational feedback and policy decisions.
5. Given several model records are available, when the model comparison section is displayed, then it shows the recorded model versions and F1 values for comparison.
6. Given no model evaluation record is available, when the Admin opens the view, then the evaluation section displays a clear unavailable state rather than fabricated metric values.
7. Given a non-admin user attempts to call the model monitoring endpoints, when the backend processes the request, then it returns an authorization error.

List

- Retrieve stored model records and evaluation metrics.
- Retrieve scan, case, response-time, and policy statistics from operational data stores.
- Separate held-out evaluation information from live operational information in the interface.
- Restrict model monitoring APIs to authenticated Admin users.

Feature `In-Application Technical Reference`

User Story 21: Read Technical Documentation in the Admin Console

US 21 As an admin, I want to read the technical documentation inside ClearURL so that I can understand the deployed design and operational interfaces without leaving the administration workspace.

Acceptance Criteria

1. Given an authenticated Admin opens the administration console, when they select `Technical docs`, then the right-side workspace changes to the technical documentation view.
2. Given the documentation asset is available, when the view loads, then the current PDF is displayed inside an embedded document frame.
3. Given the PDF is displayed, when the Admin reads it, then the document describes the system architecture, AI integration, URL analysis flow, explanation approach, API contracts, and deployment notes.
4. Given the embedded PDF viewer is insufficient for the user's browser, when the Admin selects `Open in a separate tab`, then the same PDF opens in a new tab.
5. Given the backend documentation fallback route is requested while the source PDF is missing, when the request is processed, then the backend returns a clear unavailable response rather than an unhandled error.
6. Given the Admin navigates to another Admin view, when they later return to `Technical docs`, then the embedded documentation view can be opened again without requiring a new application login.

List

- Publish the technical documentation PDF as a frontend asset.
- Embed the PDF in the Technical docs Admin workspace.
- Maintain the backend fallback document route.
- Provide a separate-tab link for browser compatibility.